Reference: Engine API

This page describes every public function and data structure in `engine/`. It states facts only. For the reasons behind the design, read `docs/explanation/tiers-explained.md`. For results and figures, read `docs/white-paper.md`.

Version: 2.0.0 (2026-10-13).

Scoring rule (all Tier 2 and Daily Double functions)

A tie for first goes to a tiebreaker clue. This rule applies to every episode since 24 November 2014. The engines return a **win share**:

Final result	Win share
Outright first place	1
k-way tie for first place	1/k
Any other place	0
Top score is $0 or less	0 for every player

Tier 1 (`fj_wager.py`) does not use this rule. Tier 1 requires an outright win.

`engine/behavior.py` — simulated contestant behavior

This module holds every number that describes contestant behavior and the scoring rule. All values are in one configuration block at the top of the file. `app/index.html` mirrors the block in its `BEHAVIOR` object. The sources and the reasons are in `docs/adr/0003-simulated-contestant-behavior.md`.

Configuration constants

Name	Value	Status	Meaning
`TIEBREAKER_RULE`	`True`	Rule since 2014-11-24	`True`: a k-way tie scores 1/k. `False`: a tie scores 1.
`FJ_ACCURACY_AVERAGE`	`0.50`	PUBLISHED	Final Jeopardy accuracy of an average contestant.
`FJ_ACCURACY_CHAMPION`	`0.60`	PUBLISHED	Final Jeopardy accuracy of a champion. No engine default uses it.
`FJ_OUTCOME_CORRELATION`	`0.30`	PUBLISHED	Correlation of right/wrong outcomes across players. Read as a binary correlation (ADR-0003).

Name	Value	Status	Meaning
`DD_ACCURACY_AVERAGE`	`0.64`	PUBLISHED	Daily Double accuracy of an average contestant.
`B_WEIGHTS`	dict	PUBLISHED for one regime, ASSUMED for five	Second-place wager mix: `bankroll`, `keepout_c`, `overtake_a`, `two_thirds`, `random`.
`A_WEIGHTS`	dict	ASSUMED	Leader wager mix when not locked out: `cover`, `cover_tie`, `bankroll`, `random`.
`A_LOCKOUT_WEIGHTS`	dict	ASSUMED	Leader wager mix in a lockout: `zero`, `max_safe`.
`RANDOM_GRID_QUARTERS`	`(0, 1, 2, 3, 4)`	ASSUMED	The "random" bet: an equal spread over 0, 1/4, 1/2, 3/4 and all of the score.
`QUAD_HALF_WIDTH`	`6.0`	Numerical	Integration range [-6, 6] for the shared clue-difficulty factor.
`QUAD_STEP`	`0.25`	Numerical	Integration step for the shared factor.

`WagerDist` is a type alias: a list of `(wager, probability)` pairs.

`win_share(final_scores, player_index) -> float`

Returns the win share of one player for a set of final scores. See "Scoring rule" above.

`wager_distribution(scores, player_index) -> WagerDist`

Returns the simulated Final Jeopardy wager distribution for one player. The function gives each player a role by score: leader (A), second (B), third (C). Equal scores go to the lower index first.

- A player with a score of $0 or less wagers $0.
- A, not locked out: uses `A_WEIGHTS`.
- A, in a lockout (A > 2B): uses `A_LOCKOUT_WEIGHTS`.
- B: uses `B_WEIGHTS`.
- C: uses the random grid only.

If a bet type does not exist in a state, the function shares its weight among the other bet types, in proportion. Each wager is clamped to [0, score]. The probabilities sum to 1.

`latent_correlation(binary_correlation) -> float`

Returns `sin(binary_correlation * pi / 2)`. This is the latent normal correlation that gives the stated binary correlation at 50% accuracy (Sheppard's formula).

`normal_cdf(x) -> float` and `inverse_normal_cdf(p) -> float`

The standard normal CDF and its inverse. `inverse_normal_cdf` uses Acklam's approximation and one Halley step. The app uses the same method.

`quadrature_nodes() -> List[Tuple[float, float]]`

Returns `(factor, weight)` pairs for the shared clue-difficulty factor. The weights sum to 1.

`conditional_accuracy(p, factor, r) -> float`

Returns the probability of a correct answer, given the shared factor, in a one-factor Gaussian copula with latent correlation `r`. If `r <= 0`, it returns `p`.

`sample_correct(ps, r, rng) -> List[bool]`

Draws one correlated right/wrong outcome per player. `ps` is the list of accuracies. `r` is the latent correlation.

`sample_wager(dist, rng) -> int`

Draws one wager from a `WagerDist`.

`engine/fj_wager.py` — Tier 1 (guarantee-based)

`legal_max_wager(score: int) -> int`

Returns the largest legal Final Jeopardy wager: the score, or $0 if the score is $0 or less.

`worst_case_ceiling(score: int) -> int`

Returns the highest score an opponent can reach: the score, doubled. This assumes that the opponent wagers everything and answers correctly. A score of $0 or less is returned unchanged.

`analyze(scores: List[int], player_index: int) -> FJRecommendation`

Computes the Tier 1 wager for one player against all others.

Parameters:

Name	Type	Meaning
`scores`	list of int	The current score of each player, in any order.
`player_index`	int	The position, in `scores`, of the player to analyze.

Returns: an `FJRecommendation` with these fields:

Field	Type	Meaning
`player_index`	int	The analyzed player.
`player_score`	int	The score of the analyzed player.
`opponent_scores`	list of int	The scores of the other players.
`lockout`	bool	`True` if no opponent can catch this player, even with an all-in correct bet.
`binding_opponent_index`	int or None	The opponent whose worst case sets the requirement. This is the single largest threat, not a combination of opponents.
`min_wager_to_guarantee_win`	int or None	The smallest wager that beats the worst case of every opponent outright. `None` if no wager can guarantee a win.
`max_legal_wager`	int	The largest legal wager (the score of the player).
`lock_tie_wager`	int or None	A wager that locks in at least a tie, when one exists.
`notes`	list of str	Plain-language notes about this result.

`analyze_all(scores: List[int]) -> List[FJRecommendation]`

Runs `analyze()` for each player. Returns one `FJRecommendation` per player, in the order of `scores`.

`engine/fj_bestresponse.py` — Tier 2 (confidence-weighted)

Tier 2 is exact in double precision. It integrates over the shared clue-difficulty factor and enumerates every wager and every right/wrong branch. It does not sample.

Module constants

Name	Value
`DEFAULT_OPPONENT_CORRECTNESS`	`behavior.FJ_ACCURACY_AVERAGE` (0.5)
`DEFAULT_CORRELATION`	`behavior.FJ_OUTCOME_CORRELATION` (0.3)

`OpponentModel`

A data structure for one opponent.

Field	Type	Meaning
`score`	int	The current score of the opponent.
`wager_dist`	`WagerDist`	Each possible wager, with its probability. The probabilities must sum to 1.0.

Field	Type	Meaning
`p_correct`	float	The probability that the opponent answers correctly. Default: `DEFAULT_OPPONENT_CORRECTNESS`.

`default_opponent_wager_model(scores, opponent_index) -> WagerDist`

Returns `behavior.wager_distribution(scores, opponent_index)`: the simulated contestant wager distribution for that opponent.

`expected_win_probability(self_score, self_wager, p_self_correct, opponents, correlation=0.0) -> float`

Returns the exact expected win share for one wager, against a list of `OpponentModel` objects.

Name	Type	Meaning
`self_score`	int	The score of the analyzed player.
`self_wager`	int	The wager to evaluate.
`p_self_correct`	float	The confidence of the analyzed player, from 0.0 to 1.0.
`opponents`	list of `OpponentModel`	The opponents.
`correlation`	float	Binary correlation of right/wrong outcomes across all players, including the analyzed player. Default: **0.0** (independent outcomes). The default is 0 so that hand-derived tests can use independent outcomes. `analyze()` uses 0.3.

`analyze(scores, player_index, p_self_correct, opponent_model_fn=default_opponent_wager_model, opponent_correctness=0.5, correlation=0.3, wager_step=100) -> FJBestResponse`

Tests candidate wagers. Returns the wager with the highest expected win share.

Parameters:

Name	Type	Meaning
`scores`	list of int	The current score of each player.
`player_index`	int	The position of the player to analyze. Raises `ValueError` if out of range.
`p_self_correct`	float	The confidence of the analyzed player, from 0.0 to 1.0.
`opponent_model_fn`	function	Makes the wager distribution of each opponent. Default: `default_opponent_wager_model`.
`opponent_correctness`	float	The accuracy of each opponent. Default: `DEFAULT_OPPONENT_CORRECTNESS` (0.5).

Name	Type	Meaning
`correlation`	float	Binary correlation of right/wrong outcomes across all players. Default: `DEFAULT_CORRELATION` (0.3).
`wager_step`	int	The gap, in dollars, between candidate wagers. Default: 100.

The candidate wagers are every multiple of `wager_step` from $0 to the legal maximum, plus the legal maximum, the Tier 1 cover wager, the cover wager minus $1, and the Tier 1 lock-tie wager, when these exist.

Returns: an `FJBestResponse` with these fields:

Field	Type	Meaning
`player_index`	int	The analyzed player.
`recommended_wager`	int	The wager with the highest expected win share. If two or more wagers are within 1e-9 of the best, the smallest wager is chosen.
`win_probability`	float	The expected win share at the recommended wager. A tie scores 1/k (see "Scoring rule").
`wager_win_probabilities`	list of (int, float)	Each candidate wager, with its expected win share.
`tier1_cover_wager`	int or None	The Tier 1 guarantee wager, for comparison.
`notes`	list of str	The tie rule, the opponent model, and the accuracy and correlation used.

`engine/simulate.py` — rollout model for the Daily Double engine

Module constants

Name	Value	Status
`DEFAULT_REGULAR_CLUE_SUCCESS_RATE`	0.6	ASSUMED
`DEFAULT_FJ_CONFIDENCE`	`behavior.FJ_ACCURACY_AVERAGE` (0.5)	PUBLISHED
`DEFAULT_FUTURE_DD_WAGER_FRACTION`	0.5	ASSUMED
`DEFAULT_FUTURE_DD_SUCCESS_RATE`	`behavior.DD_ACCURACY_AVERAGE` (0.64)	PUBLISHED

`RolloutConfig`

The assumptions for one simulated rest of the game, after a Daily Double.

Field	Type	Default	Meaning
`remaining_clue_values`	list of int	`[]`	The dollar value of each regular clue still on the board.

Field	Type	Default	Meaning
`remaining_dd_count`	int	0	The number of Daily Doubles still hidden.
`regular_clue_success_rate`	float	0.6	The chance that a player answers a given regular clue correctly.
`fj_confidence`	float	0.5	The Final Jeopardy accuracy of the analyzed player inside the rollout.
`opponent_fj_accuracy`	float	0.5	The Final Jeopardy accuracy of each opponent.
`fj_correlation`	float	0.3	Binary correlation of Final Jeopardy outcomes across all players.
`future_dd_wager_fraction`	float	0.5	The fraction of score that a player wagers on a future Daily Double.
`future_dd_success_rate`	float	0.64	The chance that a player answers a future Daily Double correctly.

`simulate_remaining_regular_clues(scores, clue_values, success_rate=0.6, rng=None) -> List[int]`

For each clue value, a uniformly random player gains the value with probability `success_rate`. Returns the new scores.

`simulate_future_daily_doubles(scores, count, wager_fraction=0.5, success_rate=0.64, rng=None) -> List[int]`

For each future Daily Double, a uniformly random player wagers `wager_fraction` of their score and wins or loses it. Returns the new scores.

`tier1_wager(scores, i) -> int`

Returns the Final Jeopardy wager of the analyzed player inside a rollout: $0 in a lockout, else the Tier 1 lock-tie wager, else the Tier 1 cover wager, else the legal maximum.

`simulate_final_jeopardy(scores, player_index, fj_confidence=0.5, opponent_accuracy=0.5, correlation=0.3, rng=None) -> List[int]`

Resolves Final Jeopardy once. The analyzed player (`player_index`) uses `tier1_wager`. Each opponent draws a wager from `behavior.wager_distribution`. Right/wrong outcomes are correlated across all players. Returns the final scores.

`wins(scores, player_index) -> float`

Returns `behavior.win_share(scores, player_index)`: 1, 1/k, or 0.

`rollout(scores, player_index, config, rng=None) -> float`

Simulates the rest of the game once: regular clues, then future Daily Doubles, then Final Jeopardy. Returns the win share of `player_index` (1, 1/k, or 0).

`engine/dd_wager.py` — Daily Double engine

`legal_max_dd_wager(score: int, round_number: int) -> int`

Returns the largest legal Daily Double wager: the greater of the score and the round cap ($1,000 in Single Jeopardy, $2,000 in Double Jeopardy).

`analyze(scores, player_index, round_number, p_correct, config=None, wager_step=100, trials_per_wager=3000, z_score=None, alpha=0.05, seed=None) -> DDRecommendation`

Runs a Monte Carlo simulation of the rest of the game for each candidate wager. All candidates use the same random numbers in each trial (common random numbers). Returns the smallest wager that the simulation cannot separate from the best one.

Parameters:

Name	Type	Meaning
`scores`	list of int	The current score of each player.
`player_index`	int	The position of the player to analyze. Raises `ValueError` if out of range.
`round_number`	int	`1` for Single Jeopardy, `2` for Double Jeopardy.
`p_correct`	float	The confidence of the analyzed player in this Daily Double clue.
`config`	`RolloutConfig` or None	The rest of the game. `None` gives `RolloutConfig()`.
`wager_step`	int	The gap, in dollars, between candidate wagers. Default: 100.
`trials_per_wager`	int	Simulated trials per candidate wager. Default: 3000.
`z_score`	float or None	A fixed band width for the plateau. `None` (default) uses a Bonferroni band, `z* = Phi^-1(1 - alpha/k)`, for `k` candidate wagers.
`alpha`	float	The family-wise error level for the Bonferroni band. Default: 0.05. The engine ignores it when `z_score` is set.
`seed`	int or None	A fixed random seed. The same seed gives the same result.

Returns: a `DDRecommendation` with these fields:

Field	Type	Meaning
`player_index`	int	The analyzed player.
`player_score`	int	The score of the analyzed player.
`round_number`	int	The round.
`p_correct`	float	The confidence used.
`recommended_wager`	int	The smallest wager on the plateau.
`win_probability`	float	The mean win share at the best candidate. This is the in-sample maximum, so it is slightly optimistic (RISK_REGISTER.md risk #11).
`wager_win_probabilities`	list of (int, float)	Each candidate wager, with its mean win share.
`standard_errors`	list of (int, float)	Each candidate's standard error, `sqrt(p(1-p)/trials)`.
`comparison_standard_errors`	list of (int, float)	Each candidate's paired standard error against the best candidate.
`plateau`	list of int	Each wager that the paired comparison cannot separate from the best one.
`z_score_used`	float	The band width used.
`trials_per_wager`	int	The trial count used.
`notes`	list of str	Plain-language caveats for this result.